Living Shoreline Meta-Analysis Model Full Workflow

George Zaragoza, Erika Y. Lin, Christen H. Fleming

2025-07-09

Last updated 2025-07-09

This workflow demonstrates how the living shoreline suitability meta-analysis model can be updated with new local data, from initial data importing and cleaning, to mapped figures of shoreline suitability. This meta-analysis model can also be used to predict the suitability and probability of success of living shoreline restoration from the geospatial data. The model and package were built for the 2024 Florida Statewide Coastal Restoration Guide (SCRG) project, and are based on the data and results of existing Living Shoreline Suitability Model (LSSM) studies. The package is maintained by the UCF Ecoinformatics Lab. Materials can be accessed at https://github.com/ecoinformatic/SCRG/.

Setup

To work with spatial data for this meta-analysis model, the `sf` and `ecoinfoscrg` R packages are needed. Other packages may also be helpful to load for data handling and visualization.

```
# If the packages are not installed, run the following:
# install.packages("sf")
# install.packages("dplyr")
# install.packages("ggplot2")
# install.packages("devtools")  # if `devtools` was not previously installed
# devtools::install_github("https://github.com/ecoinformatic/SCRG/tree/main/ecoinfoscrg")

# Load ecoinfoscrg
library(ecoinfoscrg)

# Load other requisite packages
sf::sf_use_s2(FALSE)  # for working with planar data
```

```
## Spherical geometry (s2) switched off
```

```
library(sf)  # spatial data
```

```
## Linking to GEOS 3.11.2, GDAL 3.8.2, PROJ 9.3.1; sf_use_s2() is FALSE
```

```
library(dplyr)  # data wrangling and handling
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)  # data visualization
```

Import & Clean Shapefile (.shp) Data

NOTE: *For demonstration purposes, we will use four of the same studies used to build the base meta-analysis model to show how the model can be updated. Users should use new study data collected for this purpose and do not need to include these four studies, as they are already implemented in the package.*

The `cleaningWrangling()` function automates the process of cleaning and combining data. The output is a list that includes the combined response variables and study names (`state`), the corresponding table of predictor variables (`predictors`), and vectors of all numeric variables (`numerical_vars`), all categorical variables (`categorical_vars`), and all binary variables (`binary_vars`). The arguments for `wranglingCleaning()` are `data` and `response`, which should be formatted as follows:

- `data` should be a named list of shape (shp) files, 1 file per study
- `response` is a vector of shoreline suitability response variable names, 1 per study

```
## Import shape file data
# Choctawatchee Bay data (transformed 0.001deg.shp from WGS 84 to 6346 17N )
choc <- st_read("../Data/choctawatchee_bay/choctawatchee_bay_lssm_POINTS_0.001deg.shp")

# Pensacola Bay data (transformed 0.001deg.shp from WGS 84 to 6346 17N )
pens <- st_read("../Data/santa_rosa_bay/Santa_Rosa_Bay_Living_Shoreline_POINTS_0.001deg.shp")

# Tampa Bay data (transformed 0.001deg.shp from WGS 84 to 6346 17N )
tampa <- st_read("../Data/tampa_bay/Tampa_Bay_Living_Shoreline_Suitability_Model_Results_POINTS_0.001deg

# Indian River Lagoon data (transformed 0.001deg.shp from WGS 84 to 6346 17N )
IRL <- st_read("../Data/indian_river_lagoon/UCF_livingshorelinemodels_MosquitoNorthIRL_111m.shp")

## Note that the response variable for IRL does not follow the same evaluation
## convention as the other LSSMs
# This response will need to be converted separately to levels 1-3:
IRL <- IRL %>%
  mutate(Priority = as.numeric(case_when(
    Priority %in% c(0, 1) ~ 1,
    Priority %in% c(2, 3) ~ 2,
    Priority %in% c(4, 5) ~ 3,
    TRUE ~ Priority  # keeps other values unchanged
  )))

# Rename "Erosion" variable in IRL to avoid duplicate variable names
colnames(IRL)[10] <- "Erosion_1"
```

```r
# Combine data into a named list
data <- list(choc = choc, pens = pens, tampa = tampa, IRL = IRL)

# Create vector of response variables in the order of the studies
response <- c("SMMv5Class", "SMMv5Class", "BMPallSMM", "Priority")


# Clean data using wranglingCleaning()
data <- wranglingCleaning(data = data, response = response)

# After wranglingCleaning.R
pred <- data$predictors  # store predictor data separately for easy access
state <- data$state  # store response data separately for easy access
# numerical_vars <- data$numerical_vars  # store variable names in separate vectors
categorical_vars <- data$categorical_vars
binary_vars <- data$binary_vars

# Additional manual spell check
for (var in c(categorical_vars, binary_vars)) {
  print(unique(pred[[var]]))  # check for additional inconsistencies in data entry
}

# Fix unique cases of misspellings
pred$Adj_H2[44059] <- "V"  # original spelling had an extra comma

# For binary variables not entered as Yes/No or Y/N or 1/0, change to 1/0
pred$canal <- gsub("Canal", "1", pred$canal)

# Update predictors in data object for next steps
data$predictors <- pred
```

The data is now clean and all studies have been merged into a single predictor dataset. We will now impute variables with missing values and standardize numerical variables.

Impute Missing Data & Standardize

Datasets may have missing values for certain predictor variables. For numerical and binary variables, we can use imputation techniques to fill these missing values at the site-wide scale and at the state-wide scale. Available techniques include using the median of the existing values for each variable or using the mean. Alternatively, working with spatial data offers the opportunity to use Kriging, a geostatistical method for spatial interpolation, because variables may be spatially autocorrelated. Kriging is only available at the site-wide scale, as it relies on using spatial autocorrelation for interpolation. The option to impute, and which method to use for imputation, are up to the user. However, we recommend Kriging if possible to better inform the model.

Here, we have chosen to use auto-Kriging to estimate missing values for site-wide numeric predictors where possible, and the remaining numeric predictors will be imputed with the state-wide mean values. We will also standardize our data to be centered on 0 with a standard deviation of 1, so that values are comparable across sites and variables. The mean and standard deviation used for standardization for each variable is saved to ensure that the standardization process is consistent.

```r
# Impute and standardize the data
data <- standardize(data = data, site.method = "krige", state.method = "meanImpute")
```

If Kriging (recommended), this process can take time. We can save the output as an R file (.rda), so that it can be easily accessed for subsequent tasks, without having to rerun the cleaning and standardization process each time we return to the code. Note that when RDA files are saved, the original object name will also be saved, so importing a RDA file may overwrite any same-name objects in your environment. RDS (.rds) files, though generally used to save smaller objects, can be used to reassign the object to a new name upon importing. We suggest saving such files in a separate "data" folder.

```r
# Save processed data for faster loading
save(data, file = "data/data_standardize_impute.rda")  # file path
# saveRDS(data, file = "data/data_standardize_impute.rds")  # RDS example

# Load saved data
load("data/data_standardize_impute.rda")  # imports the named "data" object
# newdata <- readRDS("data/data_standardize_impute.rda")  # importing and renaming RDS
```

Model Selection

Model selection is done by the `BUPD.R` and BUPD_nonparallel.R‘ scripts. These scripts are parallelized (for faster computation) and non-parallel, respectively. We recommend using the non-parallel script on non-Unix-based operating systems (i.e. Windows systems), as we do here. We use the ordinal probit regression for this data, to model our ordered, categorical response for shoreline suitability (categories 1, 2, 3, with 3 being the best and 1 being the worst) as a function of the other combined predictors from the LSSM studies. *Note that existing living shorelines may be categorized as unsuitable for further restoration.* AIC-based model selection is performed in a step-wise manner using the "build-up, pair-down" (BUPD) method, that evaluates the performance of models built by running through and adding predictors one at a time and then reversing the process by removing a predictor at a time, starting from the best model. The probit link function allows us to transform the response from integers of 1, 2, and 3, to a probability ranging 0-1.

Model selection via BUPD is done for each study separately, before combining the results into a meta-analysis model that will enable comparable predictions of living shoreline suitability along the entire Florida coastline. The process returns the final selected model (lowest AIC) for each site, as well as the model formula, odds ratios, and beta coefficient estimates, and the run time for the process.

```r
# Retrieve predictor data
pred <- data$predictors
pred <- pred %>%
  mutate(across(c(OBJECTID, ID, bmpCountv5, n, distance, X, Y), as.character))
## keep non predictor data as characters to avoid being selected as predictors

# List all predictors (column names of known predictors)
numeric_pred <- pred %>%
  select_if(is.numeric)
factor_pred <- pred %>%
  select_if(is.factor)
pred <- cbind(factor_pred, numeric_pred)
predictors <- colnames(pred)
```

```r
# Model selection for all 4 studies
results <- BUPD(data = data, predictors = predictors,
                parallel = FALSE)  # TRUE for faster run time if using Linux/Mac
```

```r
# Once again, we will save the results as this process can be long
save(results, file = "data/BUPD_results.rda")  # file path

# Load saved data
load("data/BUPD_results.rda")  # imports the named "results" object

str(results)  # inspect the results
```

Living Shoreline Suitability Meta-Analysis Model

Preparing for the Meta-Analysis Model

The `varCov()` function is used to organize and clean the model results for studies that are to be used in the meta-analysis model. This calls the `getBetas()` function that retrieves the beta coefficient estimates and their standard deviations, combining all the information across studies. `getBetas()` can also be used separately for coefficient retrieval from the ordinal probit regression models, however, we recommend running only `varCov()` to streamline the process.

```r
# Retrieve selected models and assign to list
mods <- list(results[[1]]$final_model,  # 1st study (Choctawatchee)
             results[[2]]$final_model,  # 2nd study (Pensacola)
             results[[3]]$final_model,  # 3rd study (Tampa)
             results[[4]]$final_model)  # 4th study (IRL)

# Retrieve betas and store in list
Betas <- list(results[[1]]$coefs,  # 1st study (Choctawatchee)
              results[[2]]$coefs,  # 2nd study (Pensacola)
              results[[3]]$coefs,  # 3rd study (Tampa)
              results[[4]]$coefs)  # 4th study (IRL)

# Retrieve betas and variance-covariance matrices for betas
VARCOV <- varCov(data = data,
                 predictors = predictors,
                 mods = mods,
                 Betas = Betas)
```

Updating the meta-analysis model with local data

The meta-analytic regression model can be updated with new data from Living Shoreline suitability assessments at local sites. This would better inform the model of the relationships between different factors and restoration suitability, which would be particularly useful for comparing suitability between different sites and studies in a standardized fashion, or for obtaining a continuous measure of the probability of success at each site (as opposed to categorized recommendations).

```r
# Build meta-analysis model
meta_analysis <- meta_regression(data = data, varCov = VARCOV)
```

Predicting from the meta-analysis model

Predicting after updating the model

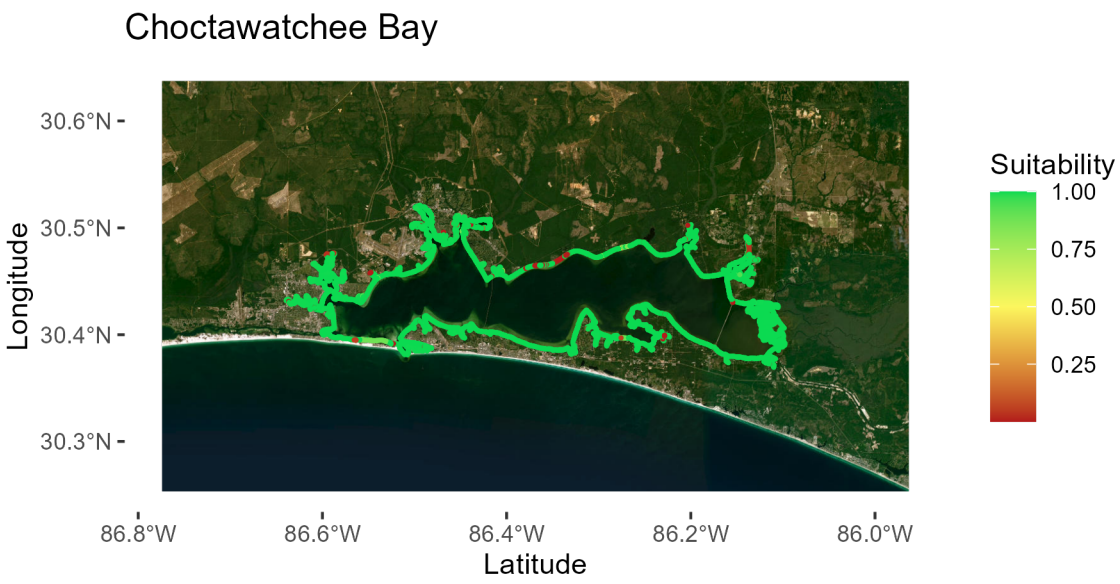
The `predict.meta()` function can be used to predict the suitability of a site for Living Shoreline restoration based on the model produced with `meta_regression()`. This requires the input of the meta-analysis model, updated with data from the sites for which suitability predictions will be made. Suitability for Living Shoreline restoration is represented as a probability for each location point, ranging 0-1 (0-100%). If no data is available, the `predict.rma()` function from the `metafor` package can help retrieve the state-wide average suitability from the model average (see "Predicting from the base state-wide model").

```
# Predict from updated model
predictions <- predict.meta_regression(data = data, meta_regression = meta_analysis)

# Add predictions to full predictor data set
pred$study <- data$state$study  # add studies back to predictors
pred$predictions <- predictions[,1]
```

Now, we can map these predictions to get a better picture of suitable locations. Remember the shapefile data we loaded at the beginning? The "geometry" column will come in handy for transforming our data into spatial features, so that we can plot our predictions on a map of the study site.

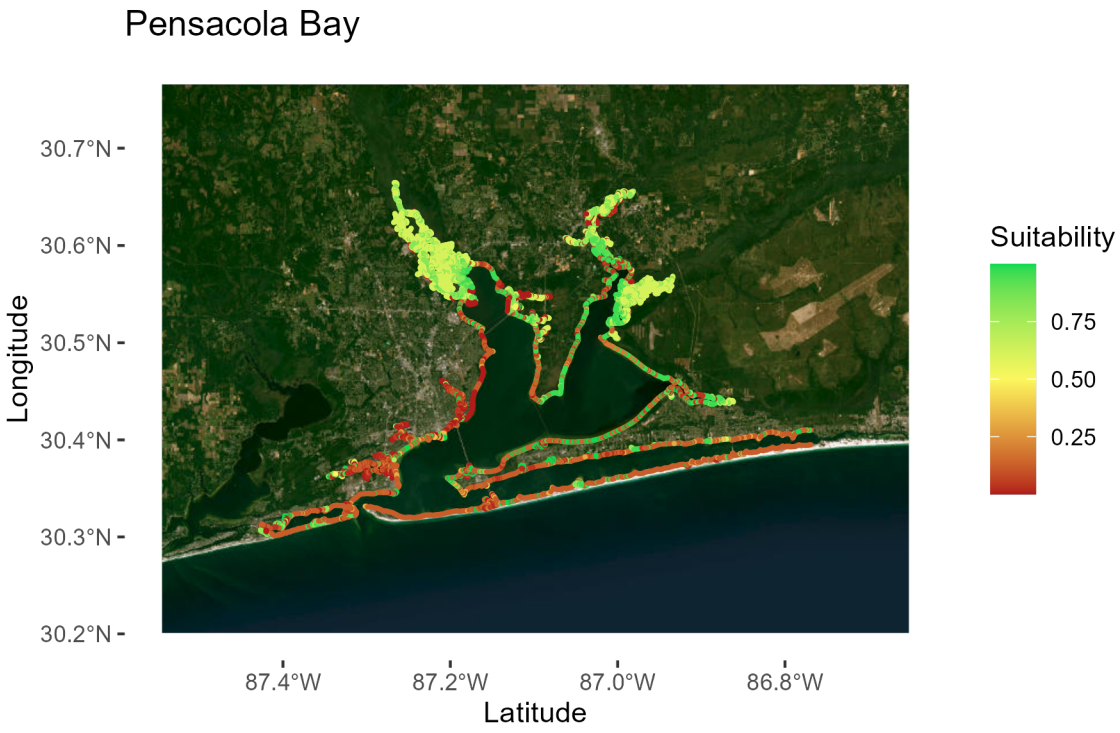
```
# Set the basemap to ESRI World Imagery
basemaps::set_defaults(map_service = "esri", map_type = "world_imagery")

## Choctawatchee Bay
# Subset full data set to Choctawatchee only
choc_pred <- pred[pred$study == "choc",]

# Obtain geometry from original shapefile
sf::st_geometry(choc_pred) <- choc$geometry  # this makes `choc_pred` a "sf" object

# Transform the coordinate reference system (CRS)
plot_choc <- sf::st_transform(choc_pred, crs = 3857)  # ensures proper projections
## The correct CRS is important for projecting properly onto the basemap

# Establish the bounding box for the map (corners of the plot)
choc_bb <- sf::st_bbox(c(xmin = -9644651-15000,
                         ymin = 3551143-15000,
                         xmax = -9584357+15000,
                         ymax = 3570713+15000),
                       crs = 3857)

# Extract the transformed coordinates
choc_coords <- as.data.frame(sf::st_coordinates(plot_choc))
plot_choc <- cbind(plot_choc, choc_coords)  # append to predictors

# Plot the predictions using `ggplot2`
choc_map <- basemaps::basemap_ggplot(ext = choc_bb) +  # extent = bounding box
  geom_sf(data = plot_choc, aes(x = X, y = Y, color = predictions), size = 0.25) +
  labs(x = "Latitude", y = "Longitude", title = "Choctawatchee Bay") +
  scale_color_gradient2(name = "Suitability", midpoint = 0.5, limits = c(0,1),
                        low = "#B31B1B", mid = "#FCF75E", high = "#0BDA51") +
  theme(plot.background = element_blank(),
```

```
        panel.background = element_blank(),
        panel.grid = element_blank(),
        legend.background = element_blank())
choc_map
```



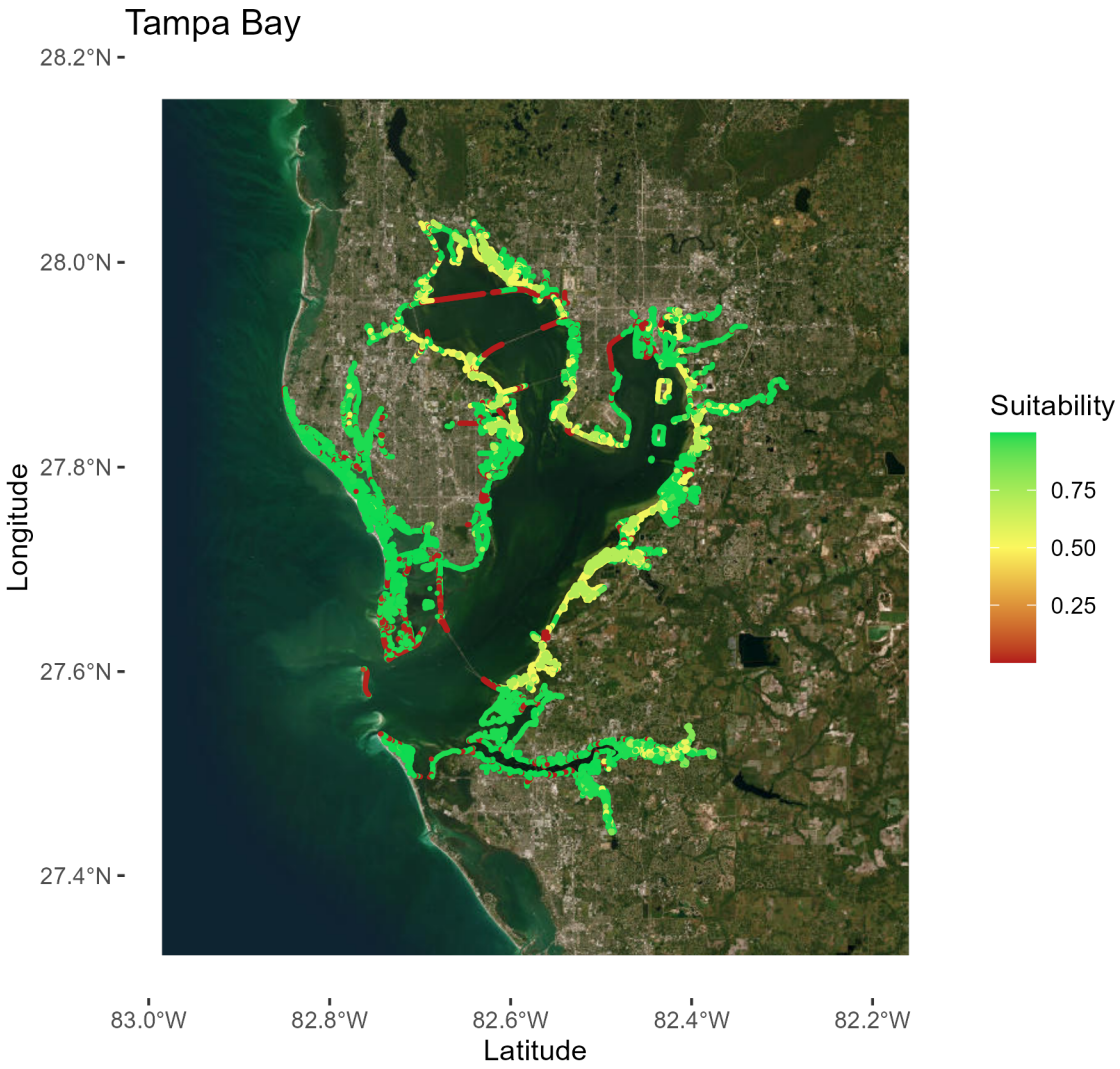

```
## Pensacola Bay
# Subset full data set to Pensacola only
pens_pred <- pred[pred$study == "pens",]

# Obtain geometry from original shapefile
sf::st_geometry(pens_pred) <- pens$geometry  # this makes `pens_pred` a "sf" object

# Transform the coordinate reference system (CRS)
plot_pens <- sf::st_transform(pens_pred, crs = 3857)  # ensures proper projections
## The correct CRS is important for projecting properly onto the basemap
```

```r
# Establish the bounding box for the map (corners of the plot)
pens_bb <- sf::st_bbox(c(xmin = -9732366-13000,
                         ymin = 3542349-13000,
                         xmax = -9659068+13000,
                         ymax = 3589286+13000),
                       crs = 3857)

# Extract the transformed coordinates
pens_coords <- as.data.frame(sf::st_coordinates(plot_pens))
plot_pens <- cbind(plot_pens, pens_coords)  # append to predictors

# Plot the predictions using `ggplot2`
pens_map <- basemaps::basemap_ggplot(ext = pens_bb) +  # extent = bounding box
  geom_sf(data = plot_pens, aes(x = X, y = Y, color = predictions), size = 0.25) +
  labs(x = "Latitude", y = "Longitude", title = "Pensacola Bay") +
  scale_color_gradient2(name = "Suitability", midpoint = 0.5, limits = c(0,1),
                        low = "#B31B1B", mid = "#FCF75E", high = "#0BDA51") +
  theme(plot.background = element_blank(),
        panel.background = element_blank(),
        panel.grid = element_blank(),
        legend.background = element_blank())
pens_map
```

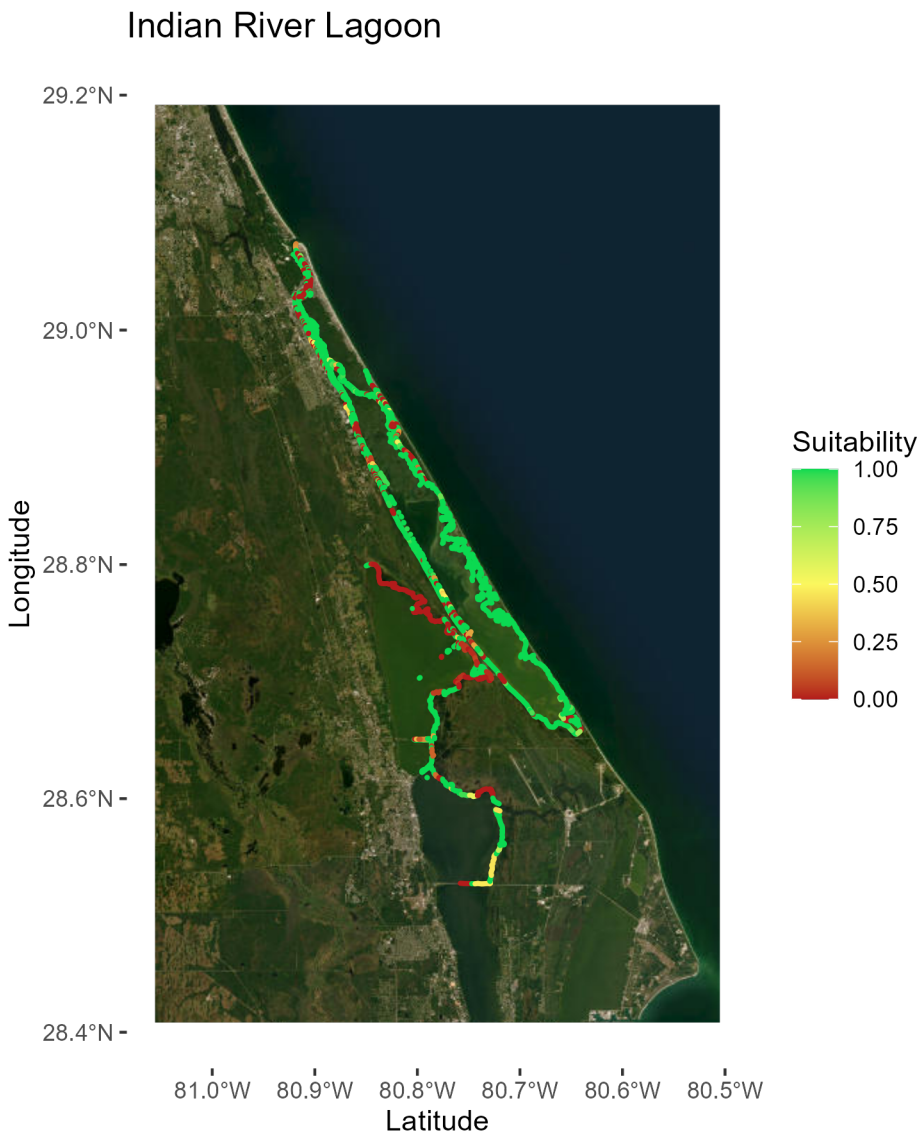



```
## Tampa Bay
# Subset full data set to Tampa only
tampa_pred <- pred[pred$study == "tampa",]

# Obtain geometry from original shapefile
sf::st_geometry(tampa_pred) <- tampa$geometry  # this makes `tampa_pred` a "sf" object

# Transform the coordinate reference system (CRS)
plot_tampa <- sf::st_transform(tampa_pred, crs = 3857)  # ensures proper projections
## The correct CRS is important for projecting properly onto the basemap

# Establish the bounding box for the map (corners of the plot)
tampa_bb <- sf::st_bbox(c(xmin = -9222837-15000,
                          ymin = 3178783-15000,
                          xmax = -9161061+15000,
                          ymax = 3253985+15000),
                        crs = 3857)
```

```r
# Extract the transformed coordinates
tampa_coords <- as.data.frame(sf::st_coordinates(plot_tampa))
plot_tampa <- cbind(plot_tampa, tampa_coords)  # append to predictors

# Plot the predictions using `ggplot2`
tampa_map <- basemaps::basemap_ggplot(ext = tampa_bb) +  # extent = bounding box
  geom_sf(data = plot_tampa, aes(x = X, y = Y, color = predictions), size = 0.25) +
  labs(x = "Latitude", y = "Longitude", title = "Tampa Bay") +
  scale_color_gradient2(name = "Suitability", midpoint = 0.5, limits = c(0,1),
                        low = "#B31B1B", mid = "#FCF75E", high = "#0BDA51") +
  theme(plot.background = element_blank(),
        panel.background = element_blank(),
        panel.grid = element_blank(),
        legend.background = element_blank())
tampa_map
```



```r
## Indian River Lagoon
# Subset full data set to IRL only
IRL_pred <- pred[pred$study == "IRL",]

# Obtain geometry from original shapefile
sf::st_geometry(IRL_pred) <- IRL$geometry  # this makes `IRL_pred` a "sf" object

# Transform the coordinate reference system (CRS)
plot_IRL <- sf::st_transform(IRL_pred, crs = 3857)  # ensures proper projections
## The correct CRS is important for projecting properly onto the basemap

# Establish the bounding box for the map (corners of the plot)
IRL_bb <- sf::st_bbox(c(xmin = -9008088-15000,
                        ymin = 3315585-15000,
                        xmax = -8976838+15000,
                        ymax = 3385010+15000),
                      crs = 3857)

# Extract the transformed coordinates
IRL_coords <- as.data.frame(sf::st_coordinates(plot_IRL))
plot_IRL <- cbind(plot_IRL, IRL_coords)  # append to predictors

# Plot the predictions using `ggplot2`
IRL_map <- basemaps::basemap_ggplot(ext = IRL_bb) +  # extent = bounding box
  geom_sf(data = plot_IRL, aes(x = X, y = Y, color = predictions), size = 0.25) +
  labs(x = "Latitude", y = "Longitude", title = "Indian River Lagoon") +
  scale_color_gradient2(name = "Suitability", midpoint = 0.5, limits = c(0,1),
                        low = "#B31B1B", mid = "#FCF75E", high = "#0BDA51") +
  theme(plot.background = element_blank(),
        panel.background = element_blank(),
        panel.grid = element_blank(),
        legend.background = element_blank())
IRL_map
```



Predicting from the base state-wide model

Currently, predictions at locations outside of the studies used to build the base meta-analysis model will return the model average as there is no information present for such sites. However, we intend to add state-wide geospatial data, such as bathymetry, to better inform the model at the state-wide scale. We will be updating the model, package, and predictions to reflect this.

```
# Use `predict.rma` from the `metafor` package
statewide_avg <- predict(meta_analysis)

# Back-transform to probabilities ranging 0-1
statewide_avg <- pnorm(statewide_avg$pred)
statewide_avg[1]

## The statewide average of the base meta-analysis model is approx. 0.25,
## meaning that sites are on average 25% suitable for living shoreline restoration
```